

Uppgiften

Bygg ett distribuerat backend-system inspirerad av hur snabbmatskedjornas ordersystem funkar, t.ex. Max eller McDonalds. Systemet ska köra med Docker Compose, vara eventdrivet och använda RabbitMQ för kommunikation mellan tjänsterna internt.

Systemet ska innehålla:

- Ett publikt HTTP-API för frontend.
- en `product-service` som ger ut produkter och menyer.
- en `order-service` som hanterar en order.
- en `kitchen-service` som tjänar köket/personalen i restaurangen.
- en `notification-service` som hanterar notiser till kunden.
- RabbitMQ
- PostgreSQL
- Automatiserade tester kopplade med CI.

Man väljer själv datastrukturer, API-kontrakt, vilka events som skickas osv.

Åtkomst

Hela systemet ska gå att starta med ett enda kommando med Docker Compose.

All trafik in i systemet ska gå genom en enda publik ingång, via `nginx` på port 80/443. Detta gör att hela systemet kommer få en enda gemensam basurl, och interna tjänster skyddas från direkt åtkomst utifrån.

Kvalitetssäkring

Ni ska visa att ni har kvalitetssäkrat systemets flöden. Här behövs det unit-testing för logikdelar man har. Det behövs end-to-end testing för ett helt flöde. Eftersom ni bestämmer flöden själv så är det erat ansvar att identifiera vad som behöver testas och kvalitetssäkras.

Testerna ska köras automatiskt vid push, antingen till main alternativt till varje bransch (genom Github Actions).

Dokumentation

Repot ska innehålla en kort README som beskriver:

- hur systemet startas,
- vilken publik väg in i systemet som används

Den behöver inte vara lång, det räcker med att läraren vet om hur man startar och testar flödet.

Muntligt försvar

Examinationen avslutas med ett individuellt muntligt försvar på cirka 10-15 min.

Ni ska kunna:

- förklara systemets arkitektur.
- motivera varför man fattat de beslut man fattat.
- beskriva hur trafiken går igenom systemet.
- nämna minst ett felscenario och hur systemet hanterar det.
- kunna svara på övriga frågor som läraren ställer om systemet.

Inlämning

Inlämning görs genom en ZIP-fil och en github länk på Canvas. Deadline för inlämningen är den 22 Juni 2026.

Betygskriterier:

För G krävs att:

- Systemet är startklar med ett enda kommando, inklusive seed och DDL.
- Systemet innehåller alla obligatoriska tjänster
- Ett fungerande flöde utan fel
- Det finns minst ett testat flöde
- Tester körs automatiskt vid kodpush (CI)
- Vid muntliga försvaret visar ni förståelse för er projekt och vad ni byggt.

För VG krävs att:

- Samtliga G-krav uppnådda
- Samtliga flöden, APIer och events är testade
- End-to-end tester från klient till avslut

- Vid muntliga försvaret visar ni en fördjupad förståelse kring er projekt
- Vid muntliga försvaret resonerar ni kring arkitekturval, vidare förbättringar, identifierar begränsningar i er lösning

Bonus

Det är meriterande att implementera ytterligare förbättringar och fördjupningar som ni identifierar som nödvändig. T.ex:

- Implementering av Redis för cache
- Mer avancerade orderflöden, till exempel betalning, avbruten order eller återbetalning.
- Swagger/OpenAPI-dokumentation.
- Error-service

Eller andra delar som ni finner nödvändiga. Ni är välkomna att improvisera och överraska.